CYPRUS INTERNATIONAL UNIVERSITY

CMPE411 Systems Programming

CHAPTER 3 - Java Concurrency

Assoc.Prof.Dr. Tamer TULGAR

Spring 2019-2020

Concurrency

Two or more sequences of events occur **"in parallel"**

- Multiprogramming
  - Single processor runs several programs at the same time
  - Each program proceeds sequentially
  - Actions of one program may occur between two steps of another
- Multiprocessors
  - Two or more processors
  - Programs on one processor communicate with programs on another
  - Actions may happen simultaneously

Process: sequential program running on a processor

The Promise of Concurrency

- Speed
  - If a task takes time t on one processor, shouldn't it take time t/n on n processors?
- Availability
  - If one process is busy, another may be ready to help
- Distribution
  - Processors in different locations can collaborate to solve a problem or work together
- Humans do it so why can't computers?
  - Vision, cognition appear to be highly parallel activities

The Challenges of Concurrency

- Concurrent programs are harder to get right
  - Folklore: need at least an order of magnitude in speedup for concurrent program to be worth the effort
- Some problems are inherently sequential
- Specific issues
  - Communication – send or receive information
  - Synchronization – wait for another process to act
  - Atomicity - do not stop in the middle and leave a mess

Language Support

- Threads (WE LEARNED IN THE PREVIOUS CHAPTER)
  - Think of a thread as a system “object” containing the state of execution of a sequence of function calls
  - Each thread needs a separate run-time stack
  - Pass threads as arguments, return as function results
- Communication abstractions
  - Synchronous communication
  - Asynchronous buffers that preserve message order
- Concurrency control
  - Locking and mutual exclusion
  - Atomicity is more abstract, less commonly provided

Inter-Process Communication

- Processes may need to communicate
  - Process requires exclusive access to some resources
  - Process need to exchange data with another process

- Can communicate via:
  - Shared variables
  - Message passing
  - Parameters

Implicit vs Explicit Concurrency

- Explicit concurrency
  - Fork or create threads / processes explicitly
  - Explicit communication between processes
    - Producer computes useful value
    - Consumer requests or waits for producer
- Implicit concurrency
  - Rely on compiler to identify potential parallelism
  - Instruction-level and loop-level parallelism can be inferred, but inferring subroutine-level parallelism has had less success

Race Conditions

- Race conditions occur when multiple, asynchronously executing threads access the same object (called a shared resource) returning unexpected (wrong) results (Processes/Threads race to gain access to the variable)
- Threads often need to share a common resource i.e. a file, with one thread reading from the file while another thread writes to the file
- They can be avoided by synchronizing the threads which access the shared resource
- May lead to Starvation
  - A Process/Thread may stay in the waiting state infinitely to gain access to the shared resource to continue processing

Critical Section

- When more than one processes/threads access a same code segment that segment is known as critical section.
- Critical section contains shared variables or resources which are needed to be synchronized to maintain consistency of data variable.
- In simple terms a critical section is group of instructions/statements or region of code that need to be executed atomically (executed without being interrupted), such as accessing a resource (file, input or output port, global data, etc.).
- In concurrent programming, if one thread tries to change the value of shared data at the same time as another thread tries to read the value (i.e. data race across threads), the result is unpredictable.
- The access to such shared variable (shared memory, shared files, shared port, etc…) to be synchronized

Critical Section

- Two concurrent processes/Threads may access a shared resource
- Inconsistent behavior if processes are interleaved
- Allow only one process in critical section
- Issues
  - How to select which process is allowed to access the critical section?
  - What happens to the other process?

Synchronization in JAVA

- If multiple threads are simultaneously trying to access the same resource strange results may occur. To overcome them java synchronization is used. The operations performed on the resource must be synchronized.

- Synchronization in java is the capability of control the access of multiple threads to any shared resource.

- Java Synchronization is better option where we want to allow only one thread to access the shared resource.

- Why we use **(Liveness)** :
    1. To prevent thread interference.
    2. To prevent consistency problem.

- **Synchronized method is used to lock an object for any shared resource.**

Synchronization in JAVA

- **Java Thread Synchronization uses**
  1. the concept of monitors
  2. the concept that every object has a lock

- **Thread synchronization is divided in two parts**
  1. Synchronized Method
  2. Synchronized Block / Statement

Synchronization in JAVA

- Monitor is the key to synchronization.
- Every object has a monitor. Strictly speaking, the monitor is not allocated unless it is used.
- A monitor allows one thread at a time to execute inside a synchronized method on the object.
- Only one thread can own a monitor at a given time. When a thread acquires a lock, it is said to have entered the monitor.
- All other threads attempting to enter the locked monitor will be suspended until the first thread exits the monitor (other threads are waiting at that time) .

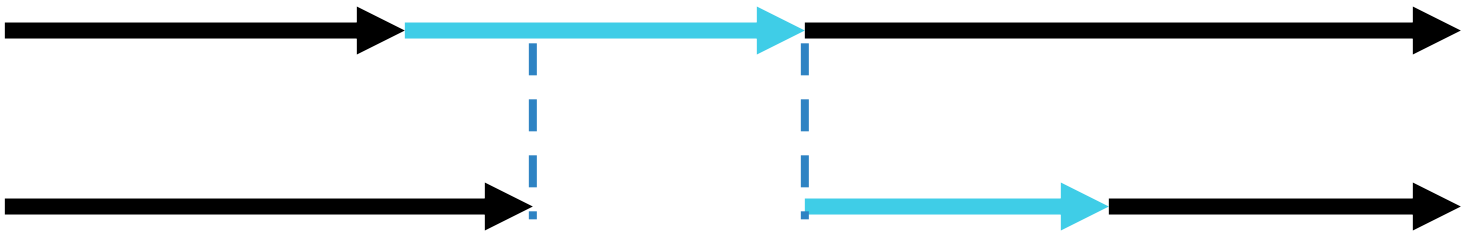
Interaction Between Threads

- Shared variables and method calls
  - Two threads may assign/read the same variable
    - Programmer is responsible for avoiding race conditions by explicit synchronization!
  - Two threads may call methods on the same object

- Synchronization primitives
  - **All objects** have an internal **lock** (inherited from **Object class**)
  - Synchronized method locks the object
    - While it is active, no other thread can execute inside object
  - Synchronization operations (inherited from Object)
    - Wait: pause current thread until another thread calls Notify
    - Notify: wake up waiting thread

Synchronized Methods

- Provide mutual exclusion
  - If a thread calls a **synchronized** method, **object is locked**
  - If another thread calls a synchronized method on the same object, this thread is blocked until object is unlocked
    - Unsynchronized methods can still be called!



- "synchronized" is <u>not</u> part of method signature
  - Subclass may replace a synchronized method with unsynchronized method

- **The synchronized methods define critical sections**
- **Execution of critical sections is mutually exclusive. Why?**

Example: Synchronized Methods

```java
class MyClass
{
    private int count;

    public MyClass()
    {
        count = 0;
    }

    public int get()
    {
        return count;
    }

    public synchronized void inc()
    {
        count++;
    }
}


class Worker implements Runnable
{
    MyClass obj;

    Worker(MyClass o) { obj = o; }

    public void run()
    {
        for(int i=0; i<1000; i++)
            obj.inc();
    }
}
```

```java
public class Updater
{
    public static void main(String[] args)
    {
        MyClass o1 = new MyClass();

        Thread t1 = new Thread(new Worker(o1));
        Thread t2 = new Thread(new Worker(o1));
        Thread t3 = new Thread(new Worker(o1));

        t1.start();
        t2.start();
        t3.start();

        try {
            t1.join();
            t2.join();
            t3.join();
        }catch(InterruptedException e){}


        System.out.println(o1.get());
    }
}
```

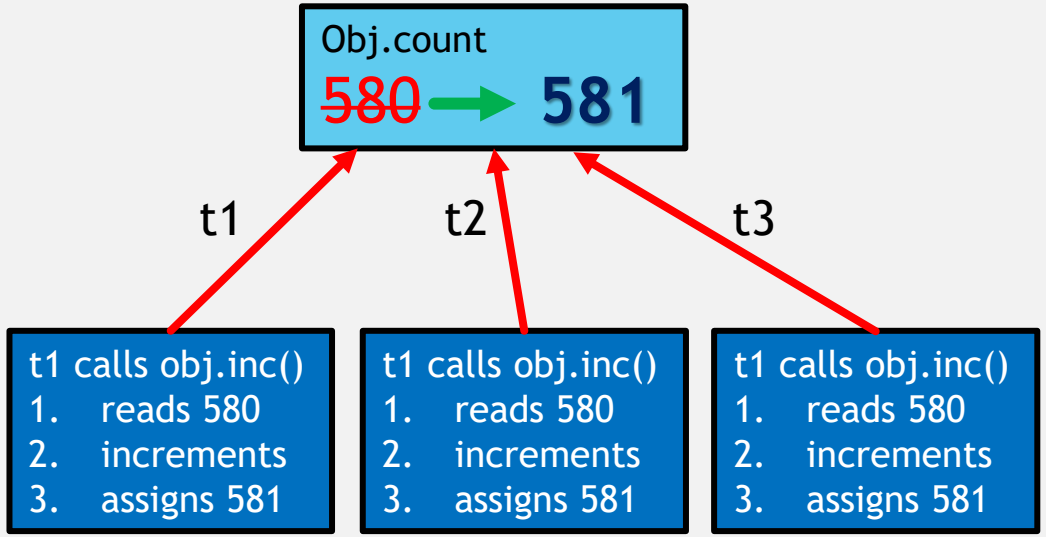
OUTPUT (As it should be)

```
3000
```

If we do not use synchronized (i.e. no synchronization is applied to our threads for the shared Myclass obj

obj.count can be read by the three threads simultaneously and all three will update the same value, overwriting each others updates

For example,



BUT THE VALUE SHOULD HAVE BEEN 583, if the concurrent access was synchronized. If un-synchronized access to count occurs multiple times, we will see e.g. 2875

Inter-thread Communication:

Methods from Object Class

```
public class Object {
      ...
      public final native void notify();
      public final native void notifyAll();

      public final native void wait(long timeout) throws InterruptedException;
      public final void wait() throws InterruptedException { wait(0); }
      public final void wait(long timeout, int nanos)
         throws InterruptedException { ... }
}
```

- We need them when threads need to co-operate
- They always work for an **object**
- May **only be called** when **object is locked**
  - Hence, **methods** which call wait(), notify(), notifyAll() should be **synchronized**
  - Typically a **condition** is also needed to decide **when wait() will be executed**.

Inter-Thread Communication - wait() method

- wait() method causes a thread to release the lock it is holding on an object; allowing another thread to run
- wait() method is defined in the Object class
- wait() can only be invoked from within synchronized code
- it should always be wrapped in a try block as it throws IOExceptions or at least InterruptedException
- wait() can only invoked by the thread that own's the lock on the object
- When wait() is called, the thread becomes disabled for scheduling and stays in waiting until one of four things occur:
  1. another thread invokes the notify() method for this object and the scheduler arbitrarily chooses to run the thread
  2. another thread invokes the notifyAll() method for this object
  3. another thread interrupts this thread
  4. the specified wait() time elapses
- When one of the above occurs, the thread becomes re-available to the Thread scheduler and competes for a lock on the object
- Once it regains the lock on the object, everything resumes as if no suspension had occurred

Inter-Thread Communication - notify() method

- Wakes up a single thread (highest priority) that is waiting on this object's monitor
  - If any threads are waiting on this object, one of them is chosen to be awakened
  - The choice is arbitrary and occurs at the discretion of the implementation
- Can only be used within synchronized code
- The awakened thread will not be able to proceed until the current thread relinquishes the lock on this object

- notifyAll() wakes up all waiting threads
  - Threads non-deterministically compete for access to object **(race-condition !!!)**
  - May not be fair (low-priority threads may never get access) **(starvation !!!)**

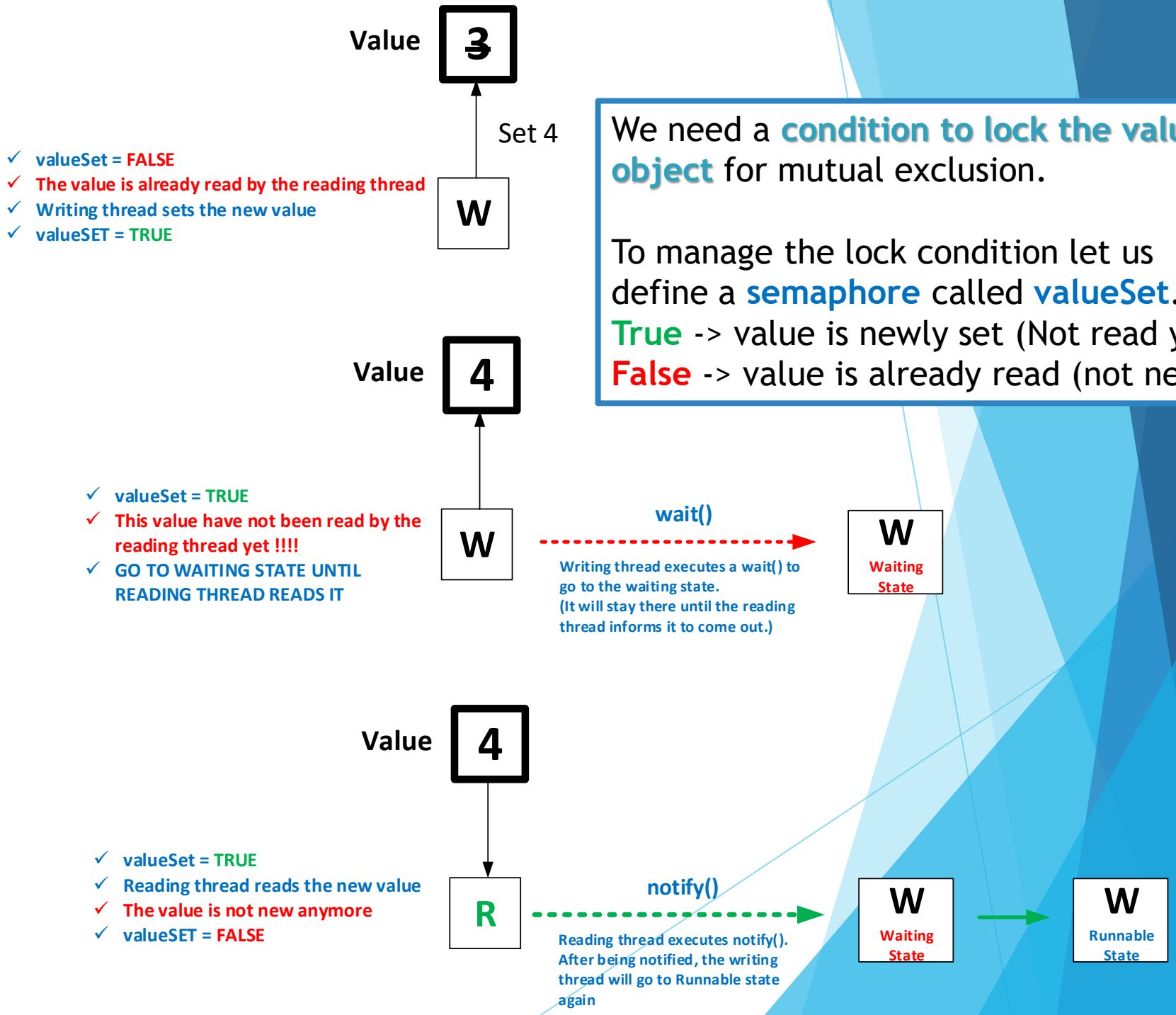
Example: Inter-Thread Communication

1. We want to have two threads, one reading(getting) from a shared object and another one writing(setting) to a shared object.

2. The reading thread should always get a new value (not previously read)

3. The writing thread should set a new value iff the reader already read the current value.

4. Hence, reader and the writer should be accessing the shared object in a synchronization.



We need a **condition to lock the value object** for mutual exclusion.

To manage the lock condition let us define a **semaphore** called **valueSet**.
True -> value is newly set (Not read yet)
False -> value is already read (not new)

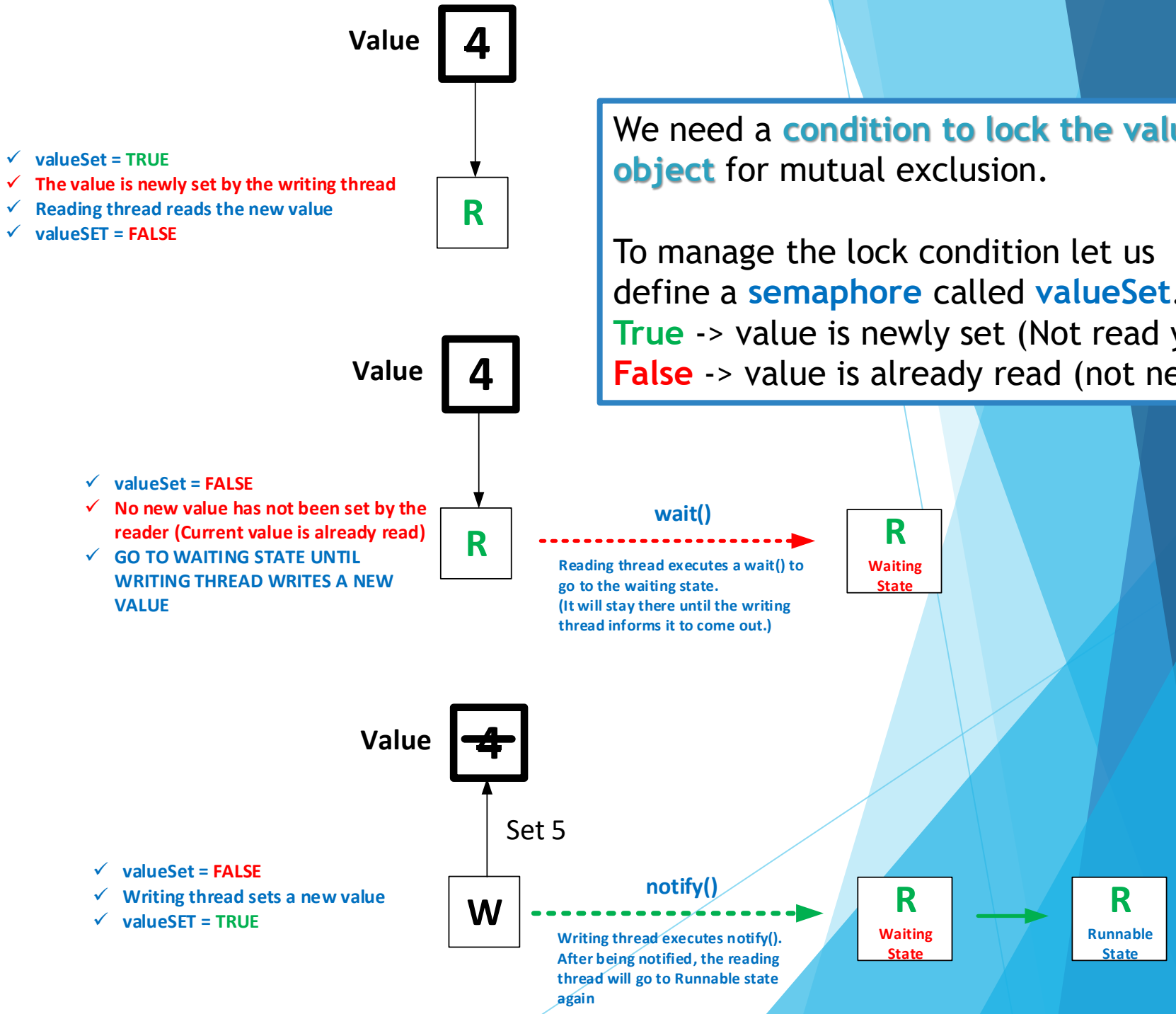
Example: Inter-Thread Communication (Continue)

1. We want to have two threads, one reading(getting) from a shared object and another one writing(setting) to a shared object.

2. The reading thread should always get a new value (not previously read)

3. The writing thread should set a new value iff the reader already read the current value.

4. Hence, reader and the writer should be accessing the shared object in a synchronization.



We need a **condition to lock the value object** for mutual exclusion.

To manage the lock condition let us define a **semaphore** called **valueSet**.
True -> value is newly set (Not read yet)
False -> value is already read (not new)

Inter-Thread Comm.

```java
class Value
{
    private int num;
    private boolean valueSet = false;

    Value() { num = 0; }

    public synchronized void set(int n)
    {
        while(valueSet)
        {
            try{ wait(); }catch(InterruptedException e){}
        }

        num = n;
        System.out.println("Set : " + num);
        valueSet = true;
        notify();
    }

    public synchronized void display()
    {
        while(!valueSet)
        {
            try{ wait(); }catch(InterruptedException e){}
        }

        System.out.println("Show : " + num);
        valueSet = false;
        notify();
    }
}
```

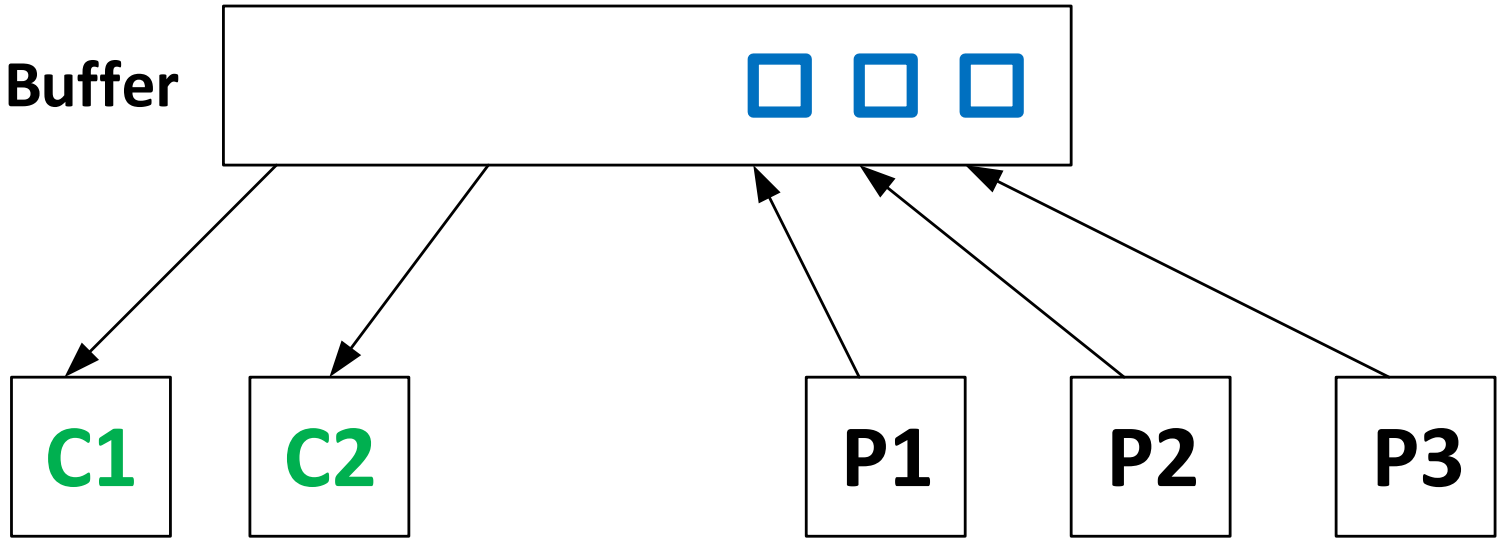
```java
class Update implements Runnable
{
    Value val;

    Update(Value v) { val = v; }

    public void run()
    {
        int i = 0;
        while(true)
        {
            val.set(i++);
            try{ Thread.sleep( millis: 1000); }catch(InterruptedException e){}
        }
    }
}
```

```java
class Read implements Runnable
{
    Value val;

    Read(Value v) { val = v; }

    public void run()
    {
        while(true)
        {
            val.display();
            try{ Thread.sleep( millis: 1000); }catch(InterruptedException e){}
        }
    }
}
```

```java
public class Wait_Notify
{
    public static void main(String[] args)
    {
        Value data = new Value();

        Thread prod = new Thread(new Update(data));
        Thread cons = new Thread(new Read(data));

        prod.start();
        cons.start();
    }
}
```

```
Set : 0
Show : 0
Set : 1
Show : 1
Set : 2
Show : 2
Set : 3
Show : 3
Set : 4
Show : 4
Set : 5
Show : 5
```

Example: Producer-Consumer Problem



We have ;

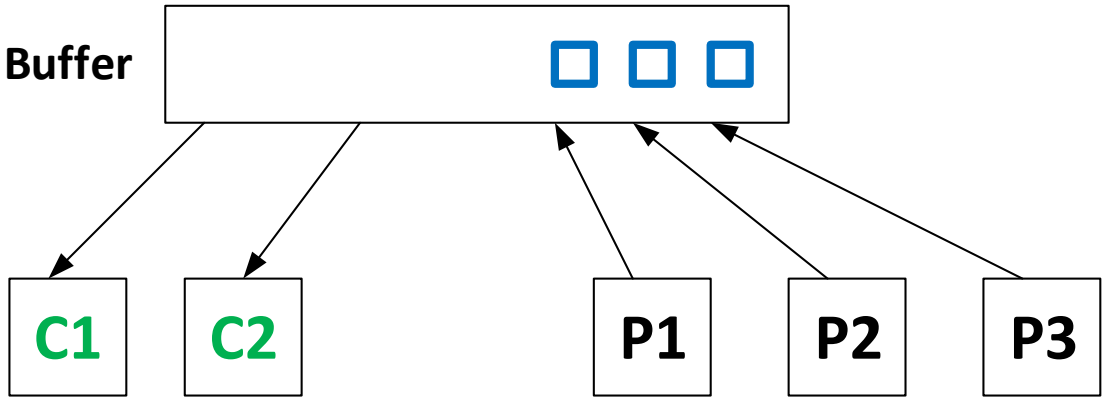
- n producers (threads which put objects into the buffer)
- m consumers (threads which take objects from the buffer)
- A buffer to maintain against overflow and underflow

Solving Producer-Consumer

- Cannot be solved with locks alone
  (Just synchronized methods will not be enough to handle ITC)
- Consumer must wait until buffer is not empty
  - For waiting, use wait method
  - Need condition recheck loop
- Producer must inform waiting consumers when there is something in the buffer
  - Must wake up at least one consumer (use notify method)

- Producer must wait until buffer is not full
  - For waiting, use wait method
  - Need condition recheck loop
- Consumer must inform waiting producers when there is empty place(s) in the buffer
  - Must wake up at least one consumer (use notify method)

Solving Producer-Consumer

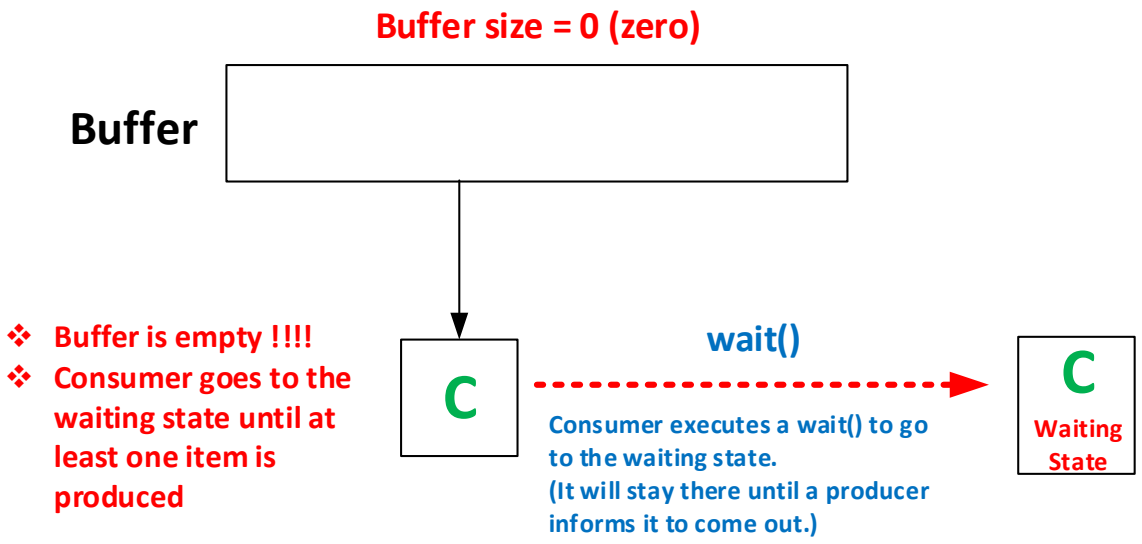



```java
import java.util.LinkedList;
import java.util.Queue;

public class MyBlockingQueue_Wait<E>
{
    private Queue<E> queue;
    private int max;


    public MyBlockingQueue_Wait(int s)
    {
        queue = new LinkedList<>();
        this.max = s;
    }
```

Let us implement our Buffer using a Queue Data Structure

- We want a queue which will accept any type of OBJECTS. Hence we use GENERICS <>
- We want a QUEUE data structure (FIFO)
- We are going to implement our Queue using a Linked List (i.e. no static size)
- Data member max defines our Buffer size

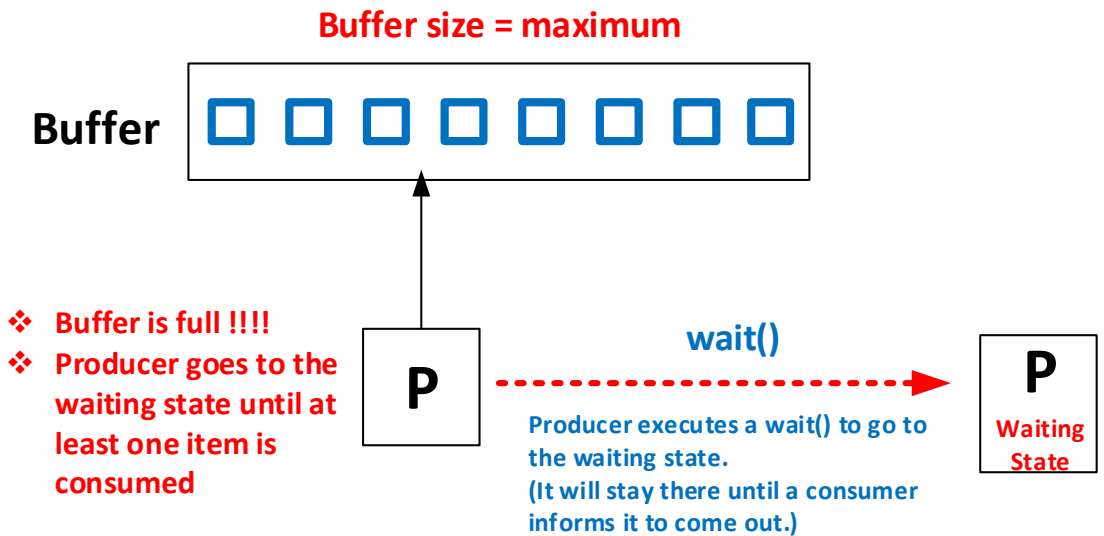
Solving Producer-Consumer



- Buffer is empty !!!!
- Consumer goes to the waiting state until at least one item is produced

For the Consumer to take an item from the Buffer, there must be at least one item in the Buffer.

(i.e. the Buffer must not be empty !!!!)



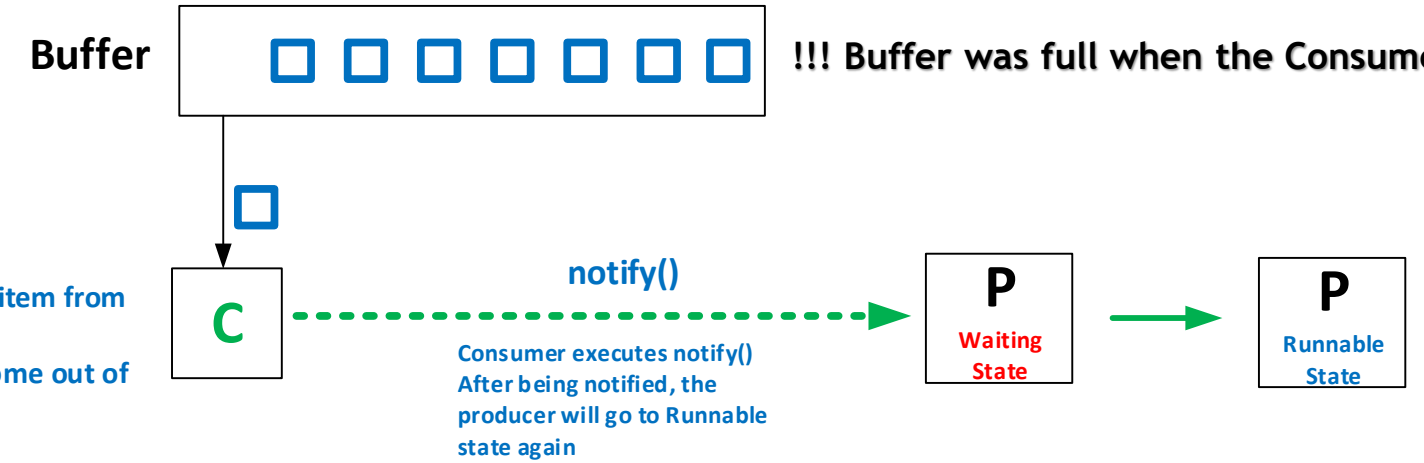
- Buffer is full !!!!
- Producer goes to the waiting state until at least one item is consumed

Likewise,

for the Producer to be able place an item into the Buffer, there must be at least one empty location in the Buffer.

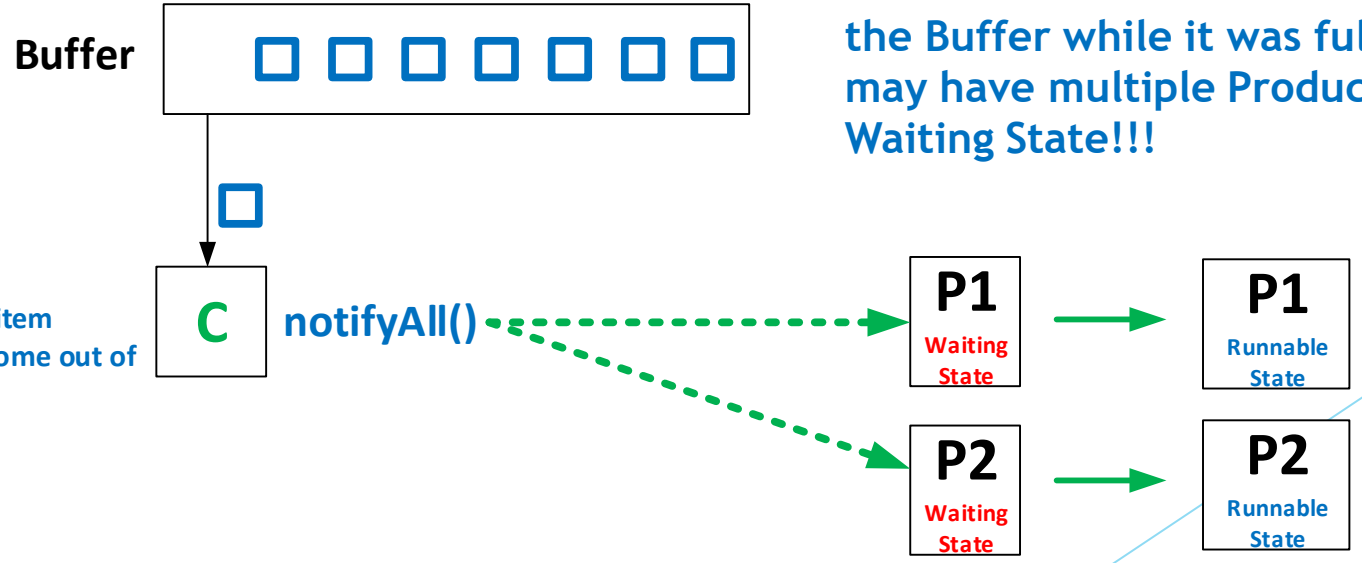
(i.e. the Buffer must not be full !!!)

Solving Producer-Consumer



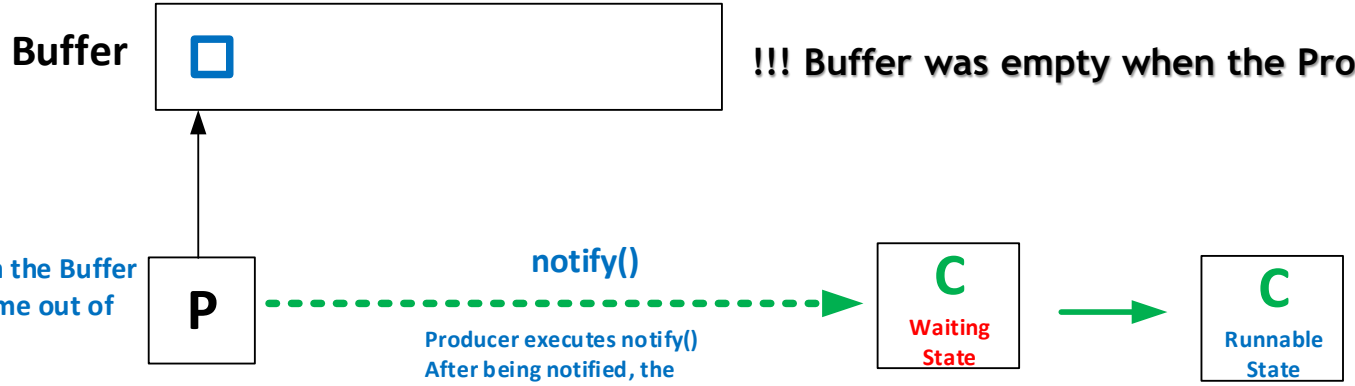
- ✓ Buffer is not empty !!!!
- ✓ Consumer consumes one item from the Buffer
- ✓ Signals the producer to come out of the Waiting state

Pay attention that we have multiple Producers. Hence, it can be the case that more than one Producers came to the Buffer while it was full and now we may have multiple Producers in the Waiting State!!!



- ✓ Buffer is not empty !!!!
- ✓ Consumer consumes one item
- ✓ Signals the producers to come out of the Waiting state

Solving Producer-Consumer

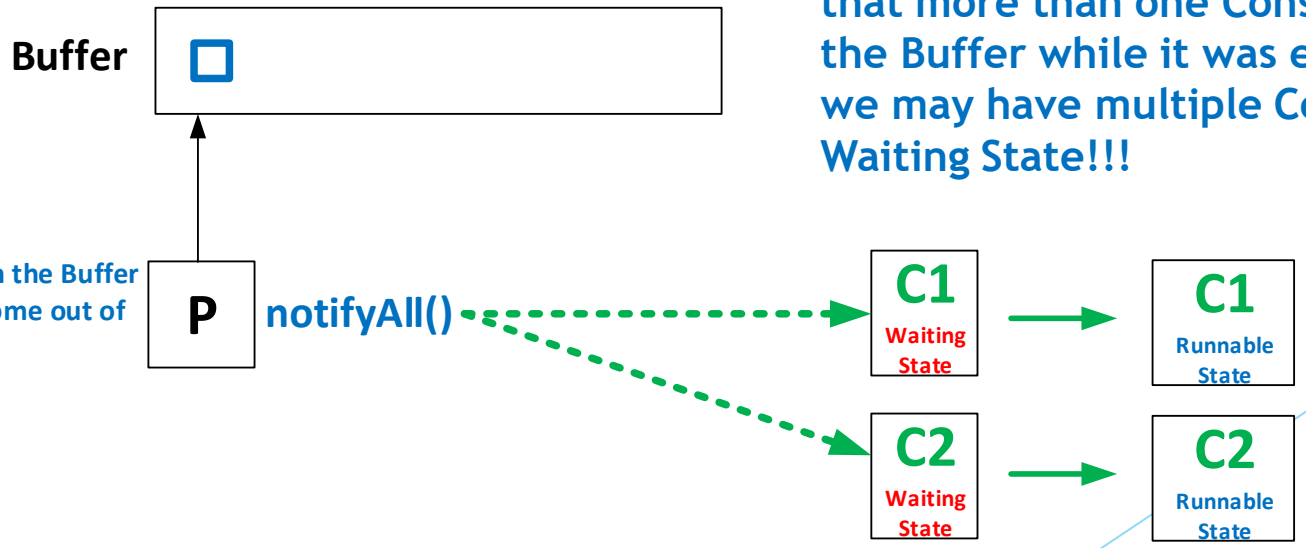


!!! Buffer was empty when the Producer arrived

- ✓ Buffer is not full !!!!
- ✓ Producer places one item in the Buffer
- ✓ Signals the consumer to come out of the Waiting state

Producer executes notify()
After being notified, the consumer will go to Runnable state again

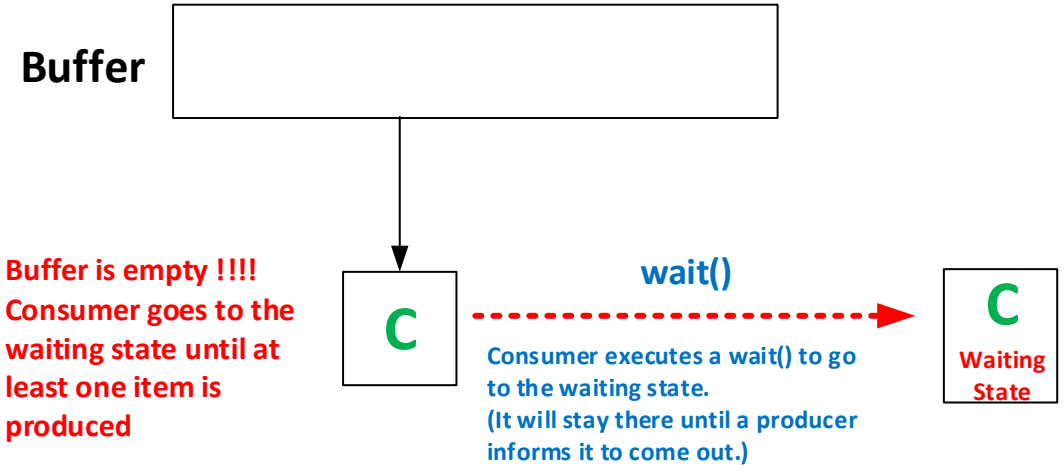
Pay attention that we have multiple Consumers. Hence, it can be the case that more than one Consumers came to the Buffer while it was empty and now we may have multiple Consumers in the Waiting State!!!



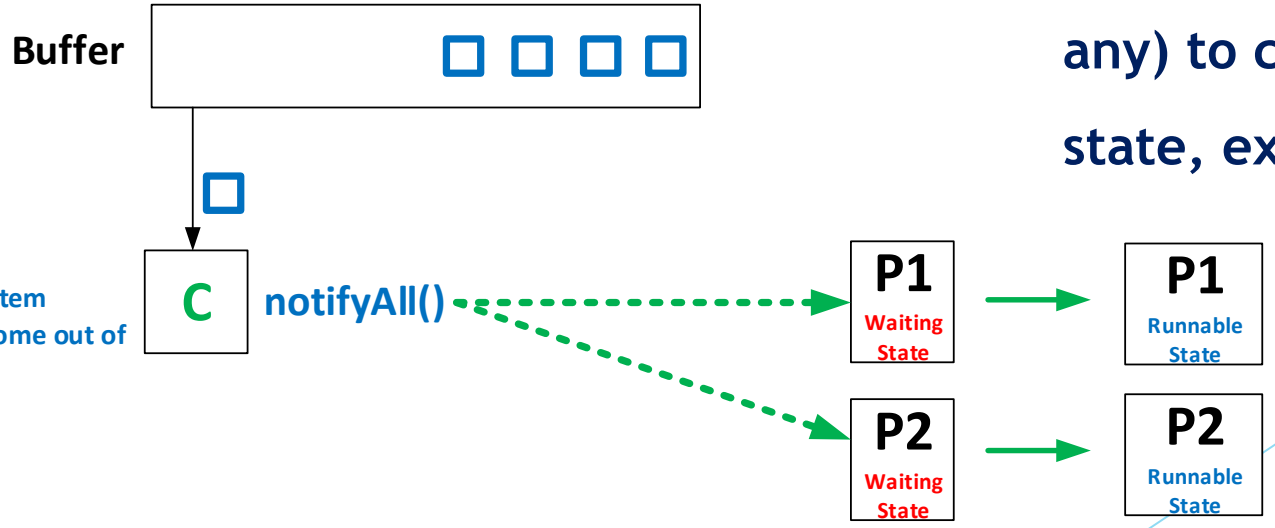
- ✓ Buffer is not full !!!!
- ✓ Producer places one item in the Buffer
- ✓ Signals the consumers to come out of the Waiting state

Solving Producer-Consumer

Buffer size = 0 (zero)



- Buffer is empty !!!!
- Consumer goes to the waiting state until at least one item is produced

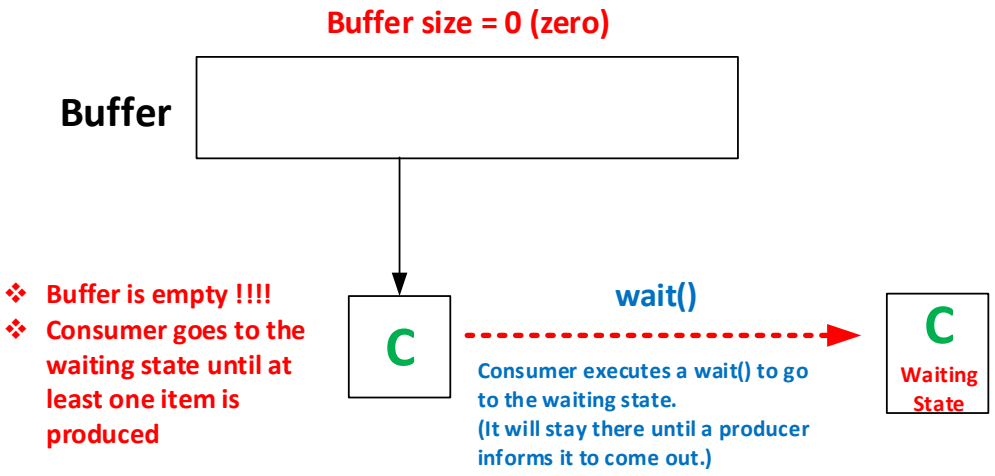


- ✓ Buffer is not empty !!!!
- ✓ Consumer consumes one item
- ✓ Signals the producers to come out of the Waiting state

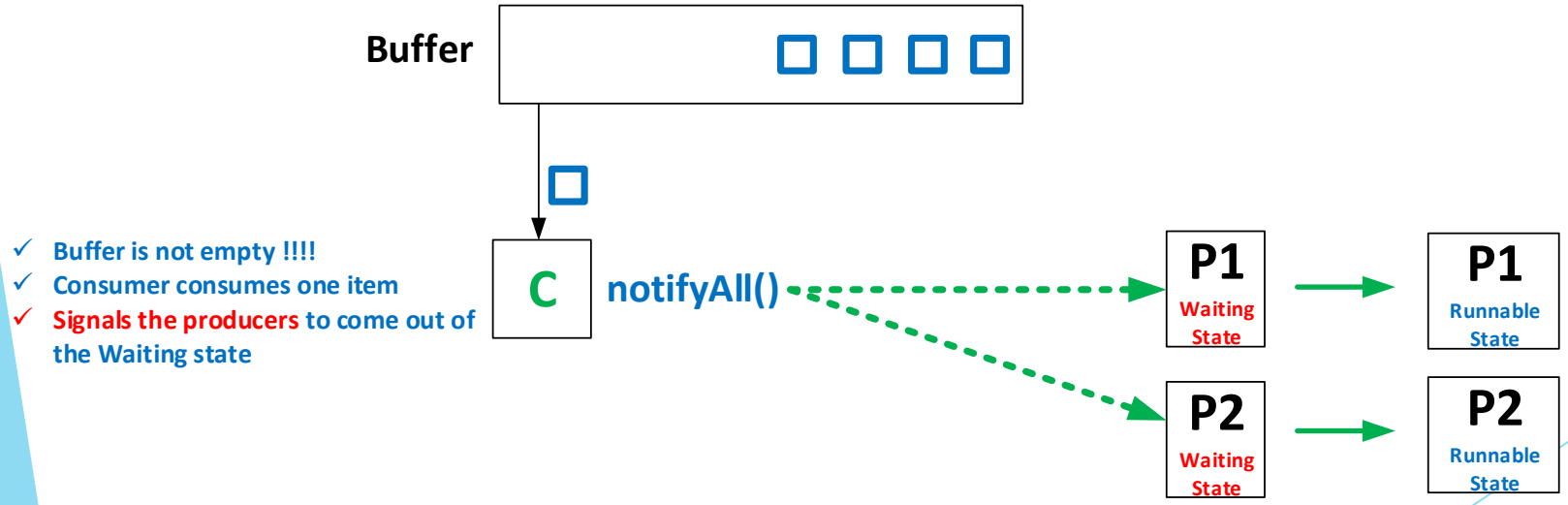
For the Consumer Thread:

1. Check if the Buffer is empty,
   i. if YES start waiting, execute wait()
   ii. If NO,
      a. consume one item
      b. Signal waiting producers (just in case ,if there are any) to come out of waiting state, execute notifyALL()

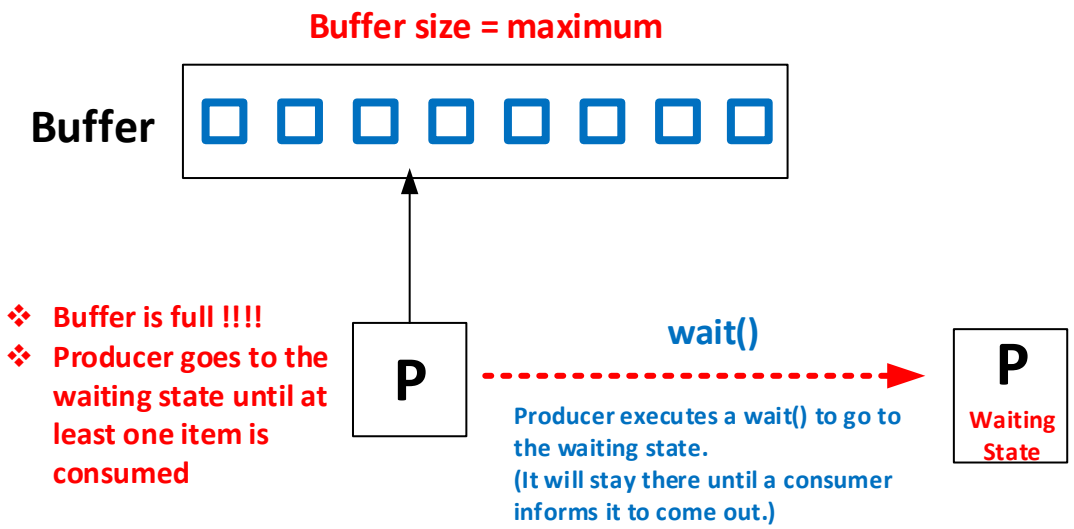
Solving Producer-Consumer




```java
public synchronized E take()
{
        while(queue.size() == 0)
        {
            // Block the thread until queue has at least 1 item to take
            try{ wait(); }catch(InterruptedException ex){}
        }

        E item = queue.remove();
        System.out.println(Thread.currentThread().getName() + " : " + item);
        notifyAll();
        return item;
}
```

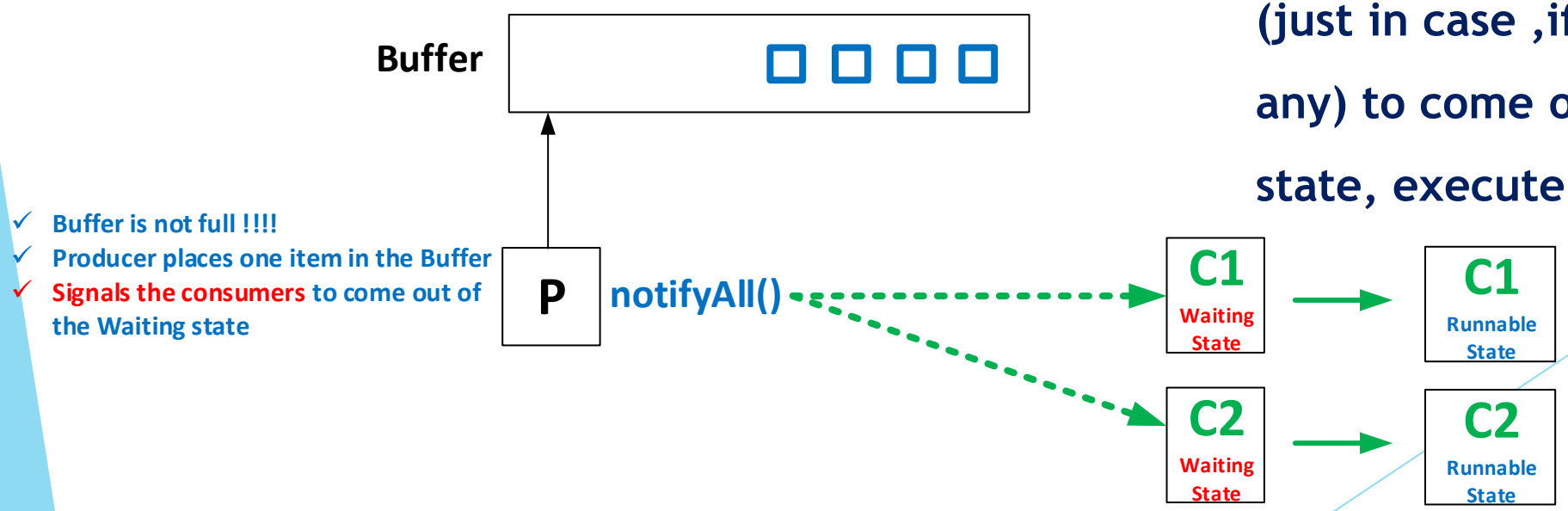


Solving Producer-Consumer

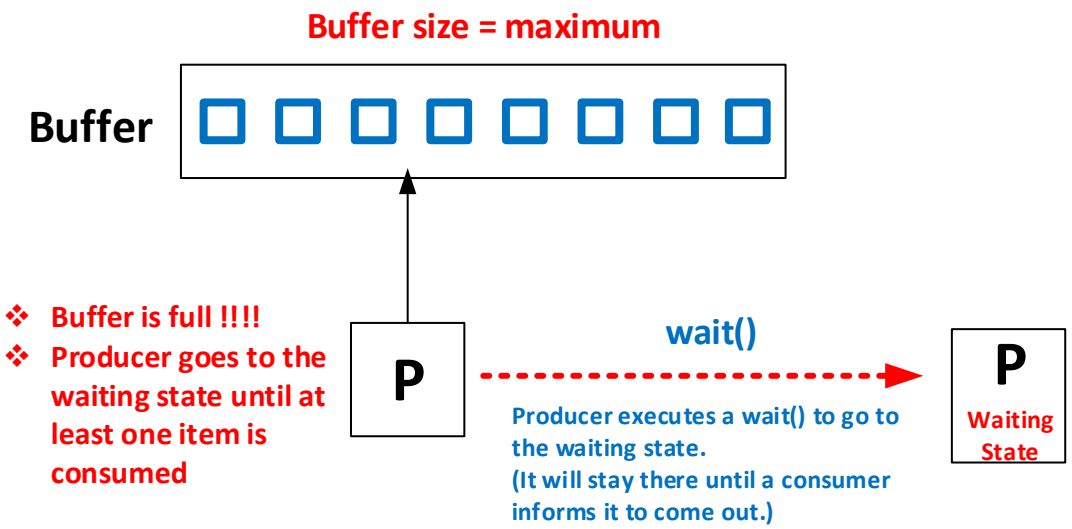


For the Producer Thread:

1. Check if the Buffer is full,
   i. if YES start waiting, execute wait()
   ii. If NO,
      a. Produce one item
      b. Signal waiting consumers (just in case ,if there are any) to come out of waiting state, execute notifyALL()

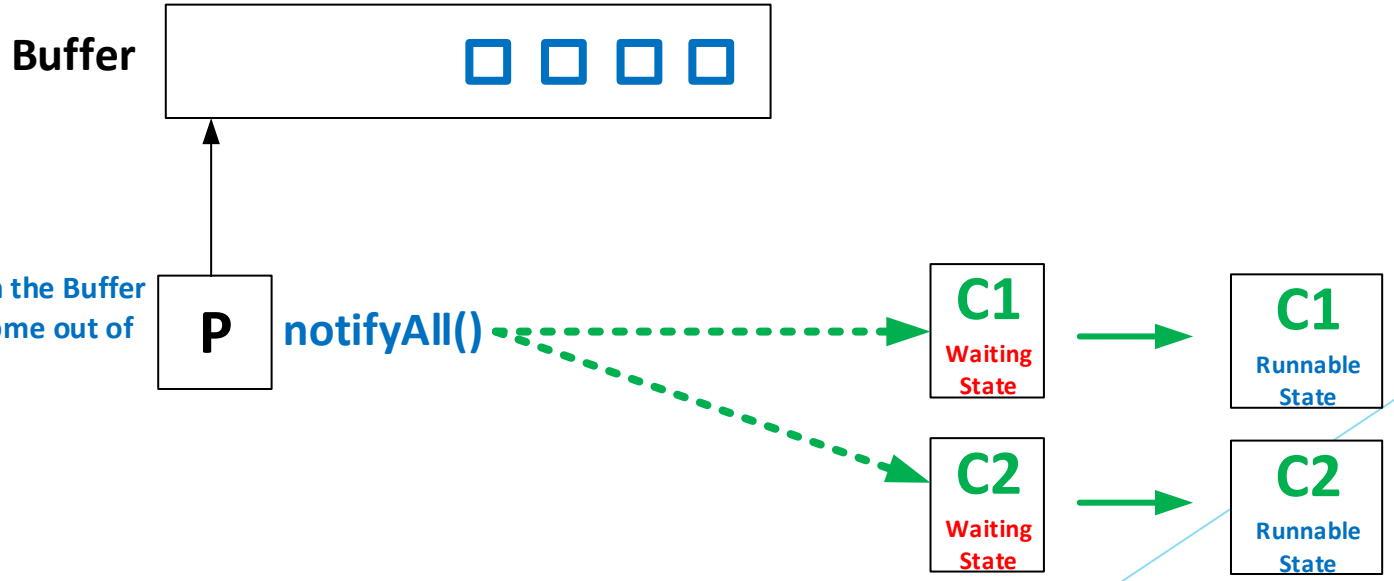


Solving Producer-Consumer



- Buffer is full !!!!
- Producer goes to the waiting state until at least one item is consumed

```java
public synchronized void put(E e)
{
        while(queue.size() == max)
        {
            // Block the thread until queue has at least 1 empty slot to add item
            try { wait(); } catch (InterruptedException ex) {}
        }
        queue.add(e);
        System.out.println(Thread.currentThread().getName() + " : " + e);
        notifyAll();
}
```



- ✓ Buffer is not full !!!!
- ✓ Producer places one item in the Buffer
- ✓ Signals the consumers to come out of the Waiting state

```java
import java.util.LinkedList;
import java.util.Queue;

public class MyBlockingQueue_Wait<E>
{
    private Queue<E> queue;
    private int max;


    public MyBlockingQueue_Wait(int s)
    {
        queue = new LinkedList<>();
        this.max = s;
    }


    public synchronized void put(E e)
    {
            while(queue.size() == max)
            {
                // Block the thread until queue has at least 1 empty slot to add item
                try { wait(); } catch (InterruptedException ex) {}
            }
            queue.add(e);
            System.out.println(Thread.currentThread().getName() + " : " + e);
            notifyAll();
    }


    public synchronized E take()
    {
            while(queue.size() == 0)
            {
                // Block the thread until queue has at least 1 item to take
                try{ wait(); }catch(InterruptedException ex){}
            }

            E item = queue.remove();
            System.out.println(Thread.currentThread().getName() + " : " + item);
            notifyAll();
            return item;
    }
```

```java
    public int size()
    {
        return queue.size();
    }
}
```

```java
class Producer implements Runnable
{
    MyBlockingQueue_Wait buffer;
    int st, sp;

    Producer(MyBlockingQueue_Wait q, int t, int p)
    {
        buffer = q;
        st = t;
        sp = p;
    }

    public void run()
    {
        for(int i = st; i<sp; i++)
            buffer.put(i);
    }
}


class Consumer implements Runnable
{
    MyBlockingQueue_Wait buffer;

    int x;

    Consumer(MyBlockingQueue_Wait q)
    {
        buffer = q;
    }

    public void run()
    {
        for(int i = 0; i < 30; i++)
            x =(Integer) buffer.take();

    }
}
```

```java
public class Prod_Cons
{
    public static void main(String[] args)
    {
        MyBlockingQueue_Wait<Integer> buf = new MyBlockingQueue_Wait<Integer>( s: 5);

        Thread p1 = new Thread(new Producer(buf,  t: 0,  p: 20), name: "Producer-1");
        Thread p2 = new Thread(new Producer(buf,  t: 20,  p: 40), name: "Producer-2");
        Thread p3 = new Thread(new Producer(buf,  t: 40,  p: 60), name: "Producer-3");

        Thread c1 = new Thread(new Consumer(buf), name: "Consumer-1");
        Thread c2 = new Thread(new Consumer(buf), name: "Consumer-2");

        p1.start();
        p2.start();
        p3.start();

        c1.start();
        c2.start();

        try {
            p1.join();
            p2.join();
            p3.join();
            c1.join();
            c2.join();
        }catch(InterruptedException e){}

        System.out.println("buffer size on exit : " + buf.size());

    }
}
```

```
Producer-2 : 20
Producer-2 : 21
Producer-2 : 22
Producer-2 : 23
Producer-2 : 24
Consumer-2 : 20
Producer-3 : 40
Consumer-1 : 21
Consumer-1 : 22
Consumer-1 : 23
Consumer-1 : 24
Consumer-1 : 40
Producer-1 : 0
Producer-1 : 1
Producer-1 : 2
Producer-1 : 3
Producer-1 : 4
Consumer-1 : 0
Consumer-1 : 1
Consumer-1 : 2
Consumer-1 : 3
Consumer-1 : 4
Producer-1 : 5
Producer-1 : 6
Producer-1 : 7
Producer-1 : 8
Producer-1 : 9
Consumer-1 : 5
Consumer-1 : 6
Consumer-1 : 7
Consumer-1 : 8
Consumer-1 : 9
Producer-1 : 10
Producer-1 : 11
Producer-1 : 12
Producer-1 : 13
Producer-1 : 14
Consumer-1 : 10
Consumer-1 : 11
Consumer-1 : 12
```

```
Producer-1 : 15
Producer-1 : 16
Producer-1 : 17
Consumer-1 : 13
Consumer-1 : 14
Consumer-1 : 15
Consumer-1 : 16
Consumer-1 : 17
Producer-3 : 41
Producer-3 : 42
Producer-3 : 43
Producer-3 : 44
Producer-3 : 45
Consumer-2 : 41
Consumer-2 : 42
Consumer-2 : 43
Consumer-2 : 44
Consumer-2 : 45
Producer-2 : 25
Producer-2 : 26
Producer-2 : 27
Producer-2 : 28
Producer-2 : 29
Consumer-2 : 25
Consumer-2 : 26
Consumer-2 : 27
Consumer-2 : 28
Consumer-2 : 29
Producer-2 : 30
Producer-2 : 31
Producer-2 : 32
Producer-2 : 33
Producer-2 : 34
Consumer-2 : 30
Consumer-2 : 31
Consumer-2 : 32
Consumer-2 : 33
Producer-2 : 35
Producer-2 : 36
Producer-2 : 37
Producer-2 : 38
```

```
Consumer-2 : 34
Consumer-2 : 35
Consumer-2 : 36
Consumer-2 : 37
Consumer-2 : 38
Producer-3 : 46
Producer-3 : 47
Producer-3 : 48
Producer-3 : 49
Producer-3 : 50
Consumer-1 : 46
Consumer-1 : 47
Consumer-1 : 48
Consumer-1 : 49
Consumer-1 : 50
Producer-1 : 18
Producer-1 : 19
Consumer-1 : 18
Consumer-1 : 19
Producer-3 : 51
Producer-3 : 52
Producer-3 : 53
Producer-3 : 54
Producer-3 : 55
Consumer-2 : 51
Consumer-2 : 52
Consumer-2 : 53
Consumer-2 : 54
Consumer-2 : 55
Producer-2 : 39
Consumer-2 : 39
Producer-3 : 56
Producer-3 : 57
Producer-3 : 58
Producer-3 : 59
Consumer-2 : 56
Consumer-2 : 57
Consumer-2 : 58
Consumer-2 : 59
buffer size on exit : 0
```